]

]

8pt

FACULTY OF TECHNOLOGY
DEPARTMENT OF ARTIFICIAL INTELLIGENCE, MACHINE LEARNING & DATA SCIENCE

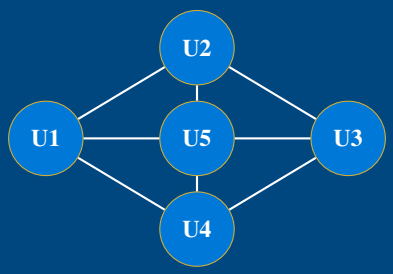
Marwadi University, Rajkot, Gujarat

DMS PROJECT REPORT

Subject Code: **01AI0402**

Discrete Mathematical Structure

Social Network Analyzer



Semester: IV Submitted On: _____

Team Details

Sr. No.	Name	Roll No.	Division
1	Vikash kumar	92460118467	4EN12
2	P.Rokhiya khanamr	92460118572	4EN12
3	K.K.Prasanna kumari	92460118581	4EN12
4	P.Ruchitha	92460118618	4EN12
5	P.Kritika	92460118606	4EN12
6	M.Vyshnavi	92460118653	4EN12
7	Delitha akula	92460118636	4EN12

CERTIFICATE

This is to certify that the project titled

"Social Network Analyzer"

has been successfully completed by the above-mentioned student(s) in partial fulfillment of the requirements for the subject **Discrete Mathematical Structure (01AI0402)**.

The project demonstrates the practical implementation of **graph theory** concepts using a web-based system developed with **HTML**, **CSS**, and **JavaScript**.

Date: _____

Dr. Apratim Dutta	**Dr. Madhu Shukla**
Faculty In-charge	*Head of Department*

Contents

Project Repository 4

1 Graph Representation 4

2 Project Motivation 5

3 Formal Definition 5

4 Types of Graphs 5

5 Key Concepts 6

6 Research Overview 6

7 Modern Applications 6

8 Research Gap 6

9 Core Functionalities 7

10 Technology Stack 7

11 Phase 1: Requirement Analysis 7

12 Phase 2: Design 7

13 Phase 3: Implementation 8

14 Phase 4: Testing 8

15 Development Workflow 8

16 System Architecture 8
- 16.1 User Interface Layer 8
- 16.2 Logic Layer 9
- 16.3 Visualization Component 9

17 System Flow 9

18 Modular Design Principle 9

19 Performance Observations 9

20 Analysis 10

21 Limitations Identified 10

22 Current Limitations 11

23 Future Work 11

24 HTML — Structure 12

25 CSS — Styling 13

26 JavaScript — Core Logic 13

><-1000

Abstract

Project Overview

Title: Social Network Analyzer **Subject:** Discrete Mathematical Structure (01AI0402)
Technology: HTML · CSS · JavaScript **Semester:** IV

The **Social Network Analyzer** is a web-based system designed to model and visualize relationships between users using graph theory concepts from discrete mathematics. In modern digital systems, understanding relationships between entities is essential, and graph-based models provide an efficient way to represent such connections.

This project focuses on developing an interactive platform where users can create **nodes** representing individuals and connect them through **edges** representing relationships. The system allows real-time visualization of networks, enabling users to understand how connections are formed and how structures evolve dynamically.

Unlike traditional theoretical approaches, this system provides a practical implementation using web technologies such as **HTML**, **CSS**, and **JavaScript**. This ensures accessibility, simplicity, and ease of use, as the application runs directly in the browser without requiring additional software or backend systems.

The main objective of this project is to bridge the gap between theoretical concepts of graph theory and their practical applications. By allowing users to interact with graph structures, the system enhances understanding and provides a foundation for advanced concepts such as:

- Network analysis
- Shortest paths
- Connectivity

Project Repository

The complete source code of this project is available on GitHub. This repository contains all implementation files including HTML, CSS, and JavaScript used in the development of the Social Network Analyzer.

GitHub Repository Link: https://github.com/vikashkumarsingh21/Social-Network-Analyzer

Introduction

Discrete Mathematics is a fundamental subject in computer science that provides the theoretical foundation for many computational systems. One of the most important topics within this field is **graph theory**, which is used to represent relationships between entities.

In real-world applications, social networks are one of the most common examples where graph theory is applied. Platforms such as **Facebook**, **Instagram**, and **LinkedIn** rely heavily on graph-based structures to manage user relationships, recommend connections, and analyze interactions.

1 Graph Representation

In a graph:

Core Graph Elements

- **Nodes** — represent users or entities
- **Edges** — represent relationships between users

2 Project Motivation

This project aims to provide a simple and interactive way to understand these concepts through a web-based system. Instead of only studying theoretical definitions, users can directly:

1. Create graphs by adding nodes
2. Connect nodes with edges
3. Visualize the resulting network structure

The system is designed to be **user-friendly** and **accessible**. It allows users to input data, create connections, and observe how networks behave. By doing so, it helps users develop a deeper understanding of graph theory and its practical applications.

Background

Graph theory is a branch of discrete mathematics that studies graphs as a mathematical representation of relationships.

3 Formal Definition

A graph is formally defined as:

Graph Definition

$$G = (V, E)$$

- V — the set of **vertices** (nodes)
- E — the set of **edges** (connections)

In the context of this project, vertices represent **users** and edges represent **relationships**.

4 Types of Graphs

Type	Description
Undirected Graph	Represents **mutual** relationships.
Directed Graph	Represents **one-way** relationships.

This project primarily uses **undirected graphs** to represent mutual connections.

5 Key Concepts

Concept	Definition
Degree	The number of connections a node has.
Path	A sequence of nodes connected through edges.
Connectivity	Determines whether nodes are reachable from one another.
Components	Subgroups (connected subgraphs) within a network.

These concepts are widely used in real-world applications such as social media platforms, communication networks, and recommendation systems.

Literature Review

Social Network Analysis (SNA) is a widely studied area in computer science and sociology. It focuses on analyzing relationships between entities using graph-based models.

6 Research Overview

Researchers have developed various techniques to understand network behavior, including:

- Centrality measures
- Clustering algorithms
- Connectivity analysis

These techniques help identify **influential nodes**, detect **communities**, and analyze **patterns** within networks.

7 Modern Applications

Modern systems leverage these concepts for a variety of purposes:

Application	Description
	Friend recommendation systems
	Content suggestion algorithms
	Network optimization

8 Research Gap

However, most existing tools are **complex** and require advanced knowledge. This project simplifies these concepts by providing a basic visualization system that allows users to understand graph structures without requiring deep

technical expertise.

Project Description

The **Social Network Analyzer** is a web-based application that enables users to create and visualize networks interactively.

9 Core Functionalities

The system allows users to:

- Add **nodes** representing individuals
- Connect nodes to form **relationships** (edges)
- **Visualize** the graph structure dynamically

10 Technology Stack

Technology	Role	Details
HTML	Structure	Defines layout, inputs, display areas
Alt CSS	Design	Styling, responsiveness, appearance
JavaScript	Logic	Graph operations, event handling

The system is designed to be **simple**, **efficient**, and **easy to use**. It provides an interactive environment where users can explore graph structures and understand relationships.

Methodology

The development of the Social Network Analyzer follows a **structured and systematic methodology** to ensure clarity, functionality, and usability.

11 Phase 1: Requirement Analysis

The methodology begins with **requirement analysis**, where the need for a system that can visually represent graph-based relationships is identified. The objective is to create a platform where users can interact with graph structures in real time without requiring advanced technical knowledge.

12 Phase 2: Design

The next step involves the **design phase**, where the user interface is planned. The focus is on simplicity and usability so that users can easily:

- Add nodes
- Create connections

- Visualize the graph

The layout is designed using **HTML** for structure and **CSS** for styling, ensuring a clean and user-friendly interface.

13 Phase 3: Implementation

During implementation, **JavaScript** is used to develop the core functionality. The logic is implemented such that user inputs directly affect the graph structure:

- When a user adds a node, it is dynamically stored and displayed.
- When a connection is created between nodes, the system updates the graph accordingly.

The system follows an **event-driven approach**, where user actions (clicking buttons, entering data) trigger specific functions that process input and update the graph in real time.

14 Phase 4: Testing

Finally, the **testing phase** ensures the system works correctly under different conditions. Various scenarios are tested, such as:

- Adding multiple nodes
- Connecting nodes
- Handling invalid inputs

This improves the reliability and performance of the system.

15 Development Workflow

Implementation & System Design

The Social Network Analyzer is implemented as a **client-side web application**, where all processing is performed within the browser. This eliminates the need for a backend server and makes the system lightweight and easy to deploy.

16 System Architecture

The system architecture consists of three main components:

16.1 User Interface Layer

Responsible for capturing user input, including:

- Input fields for node names
- Buttons for add/connect operations

- Display areas for the graph

16.2 Logic Layer

Implemented using JavaScript; acts as the **core** of the system:

- Processes user input
- Manages data structures (nodes array, edges array)
- Updates the graph on each user action

16.3 Visualization Component

Converts stored data into a visual representation:

- Nodes displayed as circles
- Edges displayed as connecting lines

17 System Flow

User Input $\longrightarrow$ **Processing** $\longrightarrow$ **Data Update** $\longrightarrow$ **Visualization**

Whenever the user interacts with the system, the graph is **re-rendered** to reflect the changes. This ensures that the output is always up to date.

18 Modular Design Principle

The design is **modular**, meaning each component works independently but contributes to the overall system. This makes the system easy to maintain and extend in the future.

Results and Analysis

The implementation of the Social Network Analyzer successfully demonstrates the practical application of graph theory concepts. The system allows users to create and visualize networks in a simple and interactive way.

19 Performance Observations

During testing, the following observations were made:

Observation	Details
Efficiency	Performs well for small to medium-sized graphs with no noticeable delay.
Dynamic Update	Graph updates immediately, providing real-time feedback.
Clarity	Nodes and edges are clearly distinguishable in the visualization.
Usability	Suitable for beginners and students due to simple interface.

20 Analysis

- Users can identify nodes with **multiple connections**, which may represent important or central entities in the network.
- **Isolated nodes** can also be observed for connectivity analysis.
- The system meets its objective of providing a simple and interactive graph visualization tool.

21 Limitations Identified

The system is limited in terms of **advanced analysis**. It does not currently include:

- ✕ Shortest path calculation
- ✕ Community detection
- ✕ Persistent data storage

These features can be incorporated in future versions.

Applications

The Social Network Analyzer has several practical applications across various fields.

Field	Application
Education	Learning tool for understanding graph theory concepts. Students can visualize nodes and edges, making it easier to grasp abstract topics.
Social Media	Demonstrates how graph-based systems model user relationships at a basic level; can be extended to analyze real-world social networks.
Data Visualization	Complex relationships between data points can be represented using graphs for easier analysis and interpretation.
Network Analysis	Studying connections between entities in communication networks, transportation systems, and organizational structures.
Advanced Systems	Serves as a foundation for more advanced tools; by adding algorithms and database integration, it can become a full-scale network analysis platform.

Limitations and Future Work

22 Current Limitations

Although the Social Network Analyzer provides a useful platform for visualizing graphs, it has certain limitations:

- **No Persistent Storage:** Since the system is entirely client-side, the data is not saved permanently. Once the page is refreshed, all data is lost.
- **Scalability:** The system works well for small graphs but may face performance issues when handling large networks with many nodes and edges.
- **Lack of Advanced Analytics:** Algorithms such as shortest path, centrality analysis, and community detection are not yet supported.
- **Basic User Interface:** The interface is functional but can be improved to provide a richer user experience.

23 Future Work

Future work can address these limitations through the following enhancements:

Enhancement	Description
Database Integration	Integrate a database to store node and edge data permanently across sessions.
Advanced Algorithms	Implement Dijkstra's algorithm (shortest path), BFS/DFS, and centrality measures.
Improved UI	Redesign the interface for better interactivity, animations, and usability.
Scalability Upgrade	Optimize rendering to handle large-scale networks efficiently.
Community Detection	Add clustering algorithms to detect communities within the network.

Code & Technical Implementation

The Social Network Analyzer is implemented using a combination of **HTML**, **CSS**, and **JavaScript**.

24 HTML — Structure

HTML defines the structure of the application, including:

- Input fields for node names and edge selection
- Buttons for triggering actions
- Display areas (canvas/container) where the graph is rendered

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Social Network Analyzer</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Social Network Analyzer</h1>
  <input type="text" id="nodeName" placeholder="Enter node name">
  <button onclick="addNode()">Add Node</button>

  <input type="text" id="nodeA" placeholder="Node A">
  <input type="text" id="nodeB" placeholder="Node B">
  <button onclick="addEdge()">Connect Nodes</button>

  <canvas id="graphCanvas" width="700" height="500"></canvas>
  <script src="app.js"></script>
</body>
</html>
```

Listing 1: HTML Structure Example

25 CSS — Styling

CSS is used to style the application and improve its appearance. It ensures that the interface is visually appealing and easy to use.

```
body {
  font-family: 'Segoe UI', sans-serif;
  background: #f0f4f8;
  text-align: center;
  padding: 20px;
}
h1 { color: #00467F; }
input {
  padding: 8px;
  margin: 5px;
  border: 1px solid #ccc;
  border-radius: 4px;
}
button {
  background: #0078D7;
  color: white;
  padding: 8px 16px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
canvas {
  border: 2px solid #00467F;
  background: #fff;
  margin-top: 20px;
}
```

Listing 2: CSS Styling Example

26 JavaScript — Core Logic

JavaScript is the most important component of the system. It handles all logic and functionality. The system uses **arrays** to store nodes and edges.

```
const nodes = [];
const edges = [];

function addNode() {
  const name = document.getElementById('nodeName').value;
  if (name && !nodes.includes(name)) {
    nodes.push(name);
    renderGraph();
```

```
  }
}

function addEdge() {
  const a = document.getElementById('nodeA').value;
  const b = document.getElementById('nodeB').value;
  if (nodes.includes(a) && nodes.includes(b)) {
    edges.push({ from: a, to: b });
    renderGraph();
  }
}

function renderGraph() {
  const canvas = document.getElementById('graphCanvas');
  const ctx = canvas.getContext('2d');
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  const positions = {};
  const r = 180;
  nodes.forEach((n, i) => {
    const angle = (2 * Math.PI * i) / nodes.length;
    positions[n] = {
      x: 350 + r * Math.cos(angle),
      y: 250 + r * Math.sin(angle)
    };
  });
  // Draw edges
  edges.forEach(e => {
    const { x: x1, y: y1 } = positions[e.from];
    const { x: x2, y: y2 } = positions[e.to];
    ctx.beginPath();
    ctx.moveTo(x1, y1);
    ctx.lineTo(x2, y2);
    ctx.strokeStyle = '#0078D7';
    ctx.lineWidth = 2;
    ctx.stroke();
  });
  // Draw nodes
  nodes.forEach(n => {
    const { x, y } = positions[n];
    ctx.beginPath();
    ctx.arc(x, y, 22, 0, 2 * Math.PI);
    ctx.fillStyle = '#00467F';
    ctx.fill();
    ctx.fillStyle = 'white';
    ctx.font = 'bold 13px Segoe UI';
```

```
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    ctx.fillText(n, x, y);
  });
}
```

Listing 3: JavaScript Logic Example

Team Contribution

The development of this project involved multiple tasks, including design, implementation, testing, and documentation.

Task	Details
UI Design	The user interface was designed using HTML and styled with CSS to ensure simplicity and ease of use.
Core Implementation	Core functionality was developed in JavaScript, including logic for node creation, edge connection, and graph visualization.
Testing	Various scenarios were tested (multiple nodes, edge connections, invalid inputs) to identify and fix issues.
Documentation	Comprehensive documentation was prepared covering methodology, implementation, and results.

Each part of the project contributes to the overall success of the system.

Conclusion

The **Social Network Analyzer** successfully demonstrates the application of graph theory concepts in a practical and interactive system. By allowing users to create and visualize networks, the project makes it easier to understand relationships between entities.

Key Achievements

- Provides a simple and effective way to explore graph structures.
- Bridges the gap between theoretical knowledge and practical implementation.
- Offers a working web-based platform accessible from any browser.
- Lays a strong foundation for further advanced development.

Although the system has certain limitations, it provides a strong foundation for further development. With additional features and improvements, it can be expanded into a more advanced **network analysis tool**.

Overall, the project achieves its objective of providing an interactive platform for understanding graph theory concepts and their practical applications in the real world.

*__References__

[1] Kenneth H. Rosen, *Discrete Mathematics and Its Applications*, 7th ed. McGraw-Hill Education.

[2] W3Schools Online Web Tutorials. *HTML, CSS, and JavaScript Documentation*. [Online]. Available: https://www.w3schools.com

[3] MDN Web Docs — Mozilla Developer Network. *Web Technology Documentation*. [Online]. Available: https://developer.mozilla.org